

A GV-drone arc routing approach for urban traffic patrol by coordinating a ground vehicle and multiple drones

Binjie Xu^a, Kexin Zhao^a, Qizhang Luo^a, Guohua Wu^{a,*}, Witold Pedrycz^b

^a School of Traffic and Transportation Engineering, Central South University, Changsha 410075, China

^b Department of Electrical and Computer Engineering, University of Alberta, Canada

ARTICLE INFO

Keywords:

Vehicle-drone coordination
Arc routing problem
Hybrid optimization algorithm
Variable neighborhood descent
Traffic patrol

ABSTRACT

With the continuous expansion of civilian field, unmanned aerial vehicle (UAV), also known as drone, has received much attention. Nevertheless, the flight range of a rotary wing drone is restricted due to limited battery endurance. To conquer this shortage, the coordination of ground vehicles (GVs) and drones to finish complex tasks has attracted much attention recently. Especially, the coordination of trucks and drones has been investigated to realize the last-mile parcel delivery. By contrast, in this study, we propose to coordinate GV and drones to monitor urban road networks more efficiently. The GV serves as a mobile carrier for launching, recovering, and charging the drones, while the drones are used to inspect some concerned sections of the road networks. To obtain high-quality routes for the GV and drones, a novel GV-drone arc routing problem (GVD-ARP) is specifically investigated. A mathematical model is constructed to describe the GVD-ARP and solved by the Gurobi optimizer. In addition, a hybrid optimization algorithm that combines the randomized variable neighborhood descent search, which is incorporated with the simulated annealing algorithm (RVND-SA), is developed to solve the model. Extensive experiments on instances with different scales are conducted and the results show the superiority of RVND-SA comparing with four other algorithms. Moreover, experiments on patrolling the real-world road networks are conducted, which further demonstrates the effectiveness of the model and RVND-SA.

1. Introduction

Traffic issues like congestion, traffic accidents, and other traffic issues affect the efficiency of urban traffic systems [1–3], and pose a great challenge for traffic management. It is therefore very meaningful to timely know about these issues by patrolling and sensing the road networks [4].

Unmanned aerial vehicles (UAVs), also known as drones, have been deployed to realize agile traffic patrols, with advantages of higher speed, less space restriction, and less pollution [5]. However, issues like limited battery endurance have affected the wide applications of drones [6]. In this case, the ground-air coordination system that consists of a fleet of ground vehicles (GVs) and drones is attracting attention [7]. In such a system, the ground vehicle can act as moving platform for landing, launching and charging the drones. The drone then can greatly extend the service range.

Previous studies on coordinated systems of GV and drones mainly focus on package deliveries [8]. However, how to coordinate GV and drones to perform road patrolling tasks is rarely investigated up to now.

The routing problem for coordinating GV and drones to perform package deliveries can be seen as an extended traveling salesman problem (TSP). By contrast, the routing problem for coordinating GV and drones to patrol road networks is essentially an extended arc routing problem, which is called the GV-drone arc routing problem in this study.

In the previous study, a heuristic algorithm was proposed to solve an arc routing problem with a GV and a drone [9]. It first generated a GV route that visited all tasks, and then tried to assign tasks to drones through a neighborhood search strategy, so as to achieve the purpose of GV-drone coordination. In this paper, we investigate a GV-drone arc routing problem (GVD-ARP) considering coordinating multiple drones and one GV to execute urban traffic patrol tasks. In the problem, the drones can perform patrol operations. As an extension of the arc routing problem, the GVD-ARP is an NP-hard problem [10]. To address this problem, we propose a hybrid optimization algorithm, which incorporates the simulated annealing algorithm into the randomized variable neighborhood descent search algorithm, abbreviated as RVND-SA. The RVND-SA searches the solution space through an improved VND search mechanism to obtain a new candidate solution

* Corresponding author.

E-mail address: guohuawu@csu.edu.cn (G. Wu).

<https://doi.org/10.1016/j.swevo.2023.101246>

Received 1 March 2022; Received in revised form 2 September 2022; Accepted 8 January 2023

Available online 10 January 2023

2210-6502/© 2023 Elsevier B.V. All rights reserved.

and selects the solution through simulated annealing mechanism and Metropolis criterion. In RVND-SA, sophisticated mechanisms such as the Metropolis criterion and four problem-specific neighborhood search strategies are appropriately incorporated to improve problem solving efficiency. We conduct extensive experiments on several instances of different scales and a practical case to test the effectiveness and applicability of the proposed algorithm. The RVND-SA is compared with four other algorithms (i.e., MA, VND, VND-SA, and RVND) in terms of total patrolling time and running time.

The remainder of this paper is organized as follows. Section 2 gives a brief overview of the relevant literature. Section 3 presents a mathematical model to describe GVD-ARP. Section 4 details the proposed algorithm, and Section 5 reports and discusses the experimental results. Section 6 gives conclusions and future research directions.

2. Related work

In this section, related works will be summarized from the following three aspects: traffic patrol, arc routing problem, and ground-air coordinated operations.

2.1. Traffic patrol

For traffic management departments, traffic patrol is an effective way to prevent and deal with traffic emergencies with limited police resources [11]. At present, common traffic patrol application scenarios include police patrol [12] and freeway service patrol [13] and other scenarios. Steil et al. [14] addressed the problem of expressing, executing, evaluating, and engaging patrol routing algorithms on road network. The routing algorithms can recommend patrol routes for organizations such as police agencies. Chawathe [15] addressed a police patrol route planning approach with minimum cost and maximum coverage of important locations as objective functions. Reis et al. [16] designed a simulation tool GAPatrol, which uses genetic algorithms to design efficient patrol routing strategies. Li et al. [17] proposed a parameter to evaluate the patrol efficiency of freeway services, which can provide a reference for traffic patrols. Du et al. [18] proposed two patrol control algorithms to improve the performance of urban traffic patrolling. All the above studies on traffic patrol routing are based on the ground vehicles as patrol platforms.

In some other studies, the drones are served as patrol platforms. Wang et al. [19] use the drones for freeway service and data collection. Wu et al. [20] and Munishkin et al. [21] put forward an information transmission model between GVs and drones and an adaptive data processing method respectively, which provide theoretical support for processing information acquired by drones during patrolling.

2.2. Arc routing problem

The research on arc routing problem (ARP) originated from

Table 1

Summary of related work on Ground-air coordinated operations to perform surveillance or patrol missions.

Research	Algorithm	Vehicles	Drones	Target type	Network
Manyam et al. [45]	MIP	1	1	Point	–
Luo et al. [46]	Heuristic	1	1	Point	Undirected
Hu et al. [42]	Heuristic	1	Multiple	Point	Undirected
Luo et al. [9]	Heuristic	1	1	Point & Arc	Directed
This Work	MIP & Metaheuristic	1	Multiple	Arc	Directed

Königsberg Seven Bridges problem proposed by Euler, and the study of modern arc routing started in 1960 with the first publication on the Chinese postman problem (CPP) [22]. Kwan [23] first raises the CPP and gives a deterministic solution method. Lima et al. [24] developed an ant colony algorithm to solve the CPP on mixed graphs, which is an NP-hard problem [25]. Another important variation of ARP is the rural postman problem (RPP), which just accesses a part of the road network [26]. Calogiuri et al. [27] studied the classical rural postman problem and solved it with a branch-and-bound algorithm. Monroy-Licht et al. [28] used the large neighborhood search algorithm to solve the rural postman problem with time windows. When considering capacity constraints, the problem becomes a capacitated arc routing problem (CARP) [29]. Huang et al. [30] considered time windows and introduced an ant colony algorithm, while Xing et al. [31] proposed an extended multi-depot CARP, which was optimized by an evolutionary approach. Cao et al. [32] constructed a multi-objective optimization model of multi-depot CARP and proposed a memetic algorithm to solve the model.

With the widespread use of drones, traditional research on ARP have been expanded. Oh et al. [33] presented a routing algorithm to solve the Chinese postman problem with drones, in which drones can pass through unconnected roads. Campbell et al. [34] first defined the drone arc routing problems (Drone ARPs). In this problem, drones can travel directly between any two points in the road network without following the edges of the network, so that an entire edge can be divided into multiple visits. The above extended studies are drone-only ARPs, and the ARPs of coordinated GVs and drones have received little attention.

2.3. Ground-air coordinated operations

Ground-air coordinated operations are attracting an increasing attention in parcel delivery, which have been extensively studied in the last few years [35]. For example, Murray and Chu [36] and Agatz et al. [37] proposed the flying sidekick traveling salesman problem (FSTSP) and the traveling salesman problem with drone (TSP-D) for the last-mile parcel delivery. These two works are about a GV (or truck) and a drone and form the basis of other extended studies [38–40]. Dorling et al. [38] applied the simulated annealing algorithm to delivery problems with a multi-trip vehicle routing problem that compensates for each drone's limited carrying capacity by reusing drones when possible. Karak and Abdelghany [41] considered the problem of pick-up and delivery at the same time. Hu et al. [42] proposed a vehicle-assisted multi-UAV inspection coordinated path scheduling algorithm, which supports the coordinated work of one GV and multiple drones. Ulmer and Thomas [43] introduced a dynamic routing problem with multiple drones and multiple GVs. Schermer et al. [44] presented an algorithm based on variable neighborhood search and tabu search to solve the extended parcel delivery problem which allows GVs to stop and launch/recovery drones at points located on the arc.

Most research on routing problem for ground-air coordinated operations to perform surveillance or patrol missions focus on target points. Manyam et al. [45] proposed a framework for cooperative vehicle routing problems for surveillance and reconnaissance missions involving a ground vehicle and an aerial vehicle. A mixed integer linear programming formulation was presented, and a branch-and-cut algorithm was developed to solve it. Luo et al. [46] investigated a two-echelon cooperated routing problem for one ground vehicle and one unmanned aerial vehicle. Hu et al. [42] proposed an algorithm referred to as a joint routing and scheduling algorithm for Vehicle-Assisted Multi-UAV inspection (VAMU), which supports the cooperation of one vehicle and multiple drones for target points in wide area inspection applications.

Nevertheless, to our knowledge, the GV-drone arc routing problem to perform surveillance and patrol missions is rarely investigated. Luo et al. [9] constructed and solved a patrol model considering one GV and one drone, in which both specific points and arcs in the urban road network

Table 2

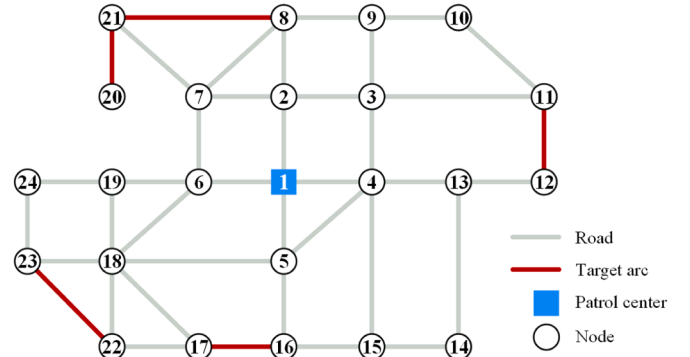
The symbol definitions.

Symbol	Meanings
G	Directed road network, $G = (V, A)$
V	Set of nodes, $V = \{1, 2, \dots, n\}$
A	Set of arcs, $A = \{(m, n) m, n \in V, m \neq n\}$
a_{mn}	An arc directed from node m to node n , $a_{mn} = (m, n)$, where $a_{mn} \in A$
TA	Set of target arcs, $TA \subseteq A$
D	Set of drones
R	Node sequence representing the GV path, of which the i^{th} element is node v_i meaning the i^{th} crucial node passed by the GV, $v_i \in V$, $ R = 2 + 2 TA $
K	Set of indexes of R , $K = \{1, 2, \dots, R \}$
K_{TR}	Subset of K , $K_{TR} = K \setminus \{1, R \}$, the GV is allowed to either launch or recover a drone at node v_i if $v_i \in K_{TR}$
K_T	Subset of K_{TR} , $K_T = K_{TR} \setminus \{ R - 1\}$, the GV is allowed to launch a drone at node v_i if $v_i \in K_T$
K_R	Subset of K_{TR} , $K_R = K_{TR} \setminus \{2\}$, the GV is allowed to recover a drone at node v_i if $v_i \in K_R$
AL	Set of all possible pairs of launch and recovery nodes in a drone flight, $AL = \{(i, j) i \in K_T, j \in K_R, i < j\}$
r_{mn}	Indicator that declares whether a_{mn} is one of target arcs. $r_{mn} = 1$ if $a_{mn} \in TE$ or $a_{mn} \in TA$, otherwise 0, $\forall m, n \in V$
c_{mn}^U	The shortest distance for drones from node m to node n , $\forall m, n \in V$
c_{mn}^{GV}	The shortest distance for the GV from node m to node n , $\forall m, n \in V$
v^{GV}	Speed of the ground vehicle
v^U	Speed of the drones
P	Maximum flying time of a drone in one flight
x_{im}	Binary variable, $x_{im} = 1$ if the i^{th} node the GV arrives at (v_i) is node $m \in V$, otherwise 0
δ_i	Continuous variable, the shortest distance the vehicle needs to travel from the i^{th} node to the $(i+1)^{th}$ node, $i \in K$, $i < R $, otherwise 0
ω_{im}	Binary variable, $\omega_{im} = 1$ if v_i and v_{i+1} are both node $m \in V$, $i \in K$, $i < R $, otherwise 0
φ_{im}^T	Binary variable, $\varphi_{im}^T = 1$ if v_i is node $m \in V$ and a drone launch here, otherwise 0
φ_{im}^R	Binary variable, $\varphi_{im}^R = 1$ if v_i is node $m \in V$ and a drone is recovered here, otherwise 0
y_{dijmn}	Binary variable, $y_{dijmn} = 1$ if drone $d \in D$ flies from $m \in V$ to $n \in V$ on a flight that takes off from v_i and lands on v_j , where $(i, j) \in AL$, otherwise 0
μ_{dij}	Binary variable, $\mu_{dij} = 1$ if drone $d \in D$ is launched and recovered on v_i and v_j , respectively, where $(i, j) \in AL$, otherwise 0
σ_{dijm}	Continuous variable, it indicates how early node $m \in V$ is visited by a drone $d \in D$ in a flight with $(i, j) \in AL$ as the launch node and recovery node.
t_i^A	Continuous variable, the time when the GV arrives at node $i \in K$
t_i^L	Continuous variable, the time when the GV leaves from node $i \in K$
t_j^D	Continuous variable, the time when a drone lands on v_j , $j \in K_R$

were simultaneously accessed. A heuristic algorithm with a simple neighborhood search strategy was proposed, and the algorithm was tested on a road network with 25 edges. In this paper, taking the total patrol time as the objective function, a hybrid optimization algorithm for solving the GVD-ARP with multiple drones and a GV is designed. A mixed integer programming (MIP) model is developed and implemented by an optimization solver after linearized. A hybrid optimization algorithm RVND-SA which combines the RVND algorithm with the acceptance criteria of the SA algorithm is proposed. The results of extensive experiments verify the MIP model and the performance of RVND-SA on different scales of instances. The comparison between the closest work with this is listed in Table 1.

3. The GV-drone arc routing problem

This section defines a two-layer arc routing problem (GVD-ARP) with multiple drones and a GV for urban traffic patrol in detail. The GVD-ARP is an extension of the ARP, with the major difference in the combination of the GV and drones. We establish a mathematical model to describe the problem.



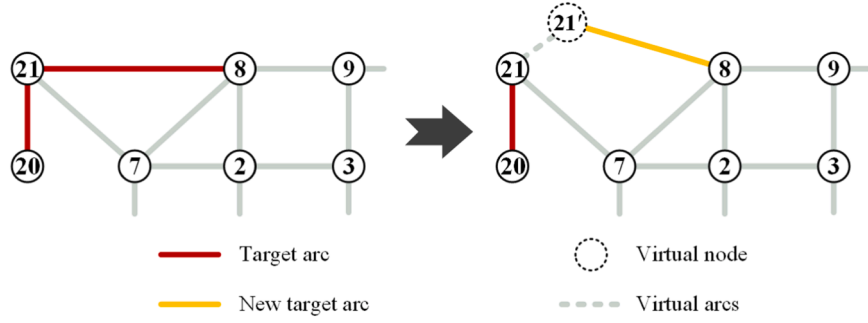


Fig. 2. Schematic diagram of the virtual node and the virtual arcs.

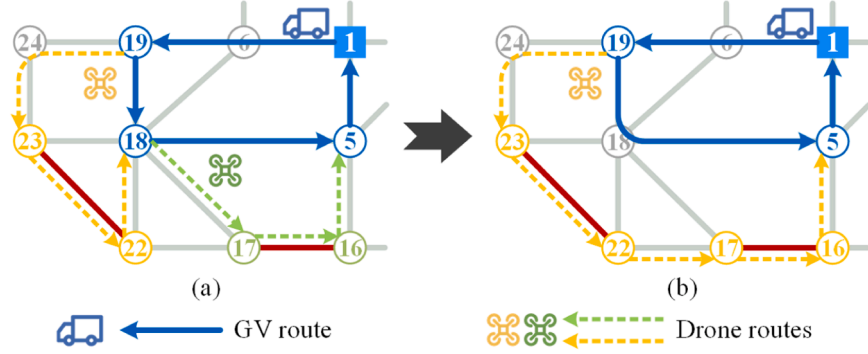


Fig. 3. Schematic diagram of the route of the GV and the drones.

The patrol center is denoted as v_{PC} . R is the sequence of nodes visited by the GV, in which each node, excluding the first and the last representing the patrol center, is a launch or recovery node of a drone. Note that even if the GV in fact launch or recover drones for times at the same node, R will record it consecutively the same number of times. A flight is a route along which a drone is launched to patrol assigned target arc(s) and then recovered by the GV, each means that R needs to record a certain pair of launch and recovery nodes. As a result, the length of R is at most $2 + 2|TE|$ when all drones patrol only one target per flight. If there are multiple targets on one flight, there will be vacancies at the end of the R which should be filled with v_{PC} .

Fig. 3 is a schematic diagram that shows two feasible solutions in a simplified instance to further explain the above definition. Two flights are performed in the solution shown in Fig. 3(a), and their launch and recovery nodes are (19, 18) and (18, 5) respectively. According to the visit sequence of the GV to these nodes, we know $R = (1, 19, 18, 18, 5, 1)$, where node 18 appears twice because it is both the recovery node of the first flight and the launch node of the second flight. Assuming the drone has enough charge, the quality of this solution may be improved by merging the two flights, as shown in Fig. 3(b). In this case, the launch and recovery nodes of the flight are (19, 5), and R becomes (1, 19, 5, 1) which gets shorter due to the reduction in the number of flights. Node 18 does not appear because it is just a passing node and does not involve any launch or recovery of drones any more. To keep the length of R constant for modeling and coding purposes, R will become (1, 19, 5, 1, 1, 1) by attaching enough v_{PC} s to its end.

Between two adjacent nodes in R , The GV travels along the shortest path solved by the Floyd-Warshall algorithm. The shortest distance for the GV and drones between node m and n is represented by c_{mn}^{GV} and c_{mn}^U respectively, since the GV is restricted by one-way streets and the drone is not.

3.2. Objective function

The objective of the optimization problem is to minimize the total

time required for the GV and the drones coordinately finish all road section patrol tasks and return to the patrol center. Let t_i^A and t_i^L denote the time when the GV arrives at and leave from node v_i respectively. t_i^L is larger than t_i^A only if the GV recovers a drone and arrives earlier than the drone at node v_i , otherwise t_i^L is equal to t_i^A . v_1 and $v_{|R|}$ in R only represent the start and end node of the GV path, respectively, where no launch or recovery of drones, so $t_1^A = t_1^L$ and $t_{|R|}^A = t_{|R|}^L$.

As Eq. (1) shows, the objective function is the total time taken by the GV and the drones to return to the patrol center after completing all the tasks, i.e., the time $t_{|R|}^A$ when the $|R|^{th}$ node the GV arrives at.

$$\min t_{|R|}^A \quad (1)$$

3.3. Constraints

Let a binary variable $x_{im} = \{0, 1\}$ represents whether the i^{th} node the GV arrives at is node $m \in V$. For the GV, the following constraints need to be met.

$$x_{i',1} + x_{i'',1} = 2, \quad i' = 1, \quad i'' = |R| \quad (2)$$

$$\sum_{m \in V} x_{im} = 1, \quad \forall i \in K \quad (3)$$

Constraint (2) indicates that the first and last node the GV arrives at must be the patrol center which is assumed to be represented by node 1. Constraint (3) ensures that each node the GV visit has to be exactly one of nodes in V .

In a drone flight, the launch node should be visited by the GV earlier than the recovery node, and neither is the first or last one in R . Based on this, all possible combinations of launch and recovery nodes in R constitute the set AL . The binary variable $y_{dijmn} = 1$ if drone $d \in D$ flies from $m \in V$ to $n \in V$ on a flight that takes off from v_i and lands on v_j , where $(i, j) \in AL$. Note that it is not required that node m and n must be directly connected by an arc in A , but can be any nodes in the graph G .

The drone travels along the shortest path between m and n ignoring the limit of one-way streets. Based on the definitions above, the routes of drone flights must meet the following constraints.

$$\sum_{d \in D} \sum_{(i,j) \in AL} (y_{dijmn} + y_{dijnm}) \geq r_{mn}, \forall m, n \in V \quad (4)$$

$$\sum_{m \in V} y_{dijmn} \leq 1, \forall d \in D, \forall (i,j) \in AL, \forall n \in V \quad (5)$$

$$\sum_{m \in V} y_{dijnm} \leq 1, \forall d \in D, \forall (i,j) \in AL, \forall n \in V \quad (6)$$

Constraint (4) indicates that every target arc needs to be visited either forward or reverse by drones once at least. Constraints (5) and (6) prevent one drone visit a node multiply times in a flight. There may have been multiple visits by a drone to some node to seek the shortest distance if the node is shared by multiple target arcs patrolled in a flight, while the virtual nodes and virtual arcs mentioned in Section 3.1 can keep these two constraints still meet.

$$\sum_{j \in K_R, j > i} y_{dijmn} \leq 1, \forall d \in D, \forall i \in K_T, \forall m, n \in V \quad (7)$$

$$\sum_{i \in K_T, i < j} y_{dijmn} \leq 1, \forall d \in D, \forall j \in K_R, \forall m, n \in V \quad (8)$$

$$\sum_{m \in V} y_{dijmm} = 0, \forall d \in D, \forall (i,j) \in AL \quad (9)$$

Constraints (7) and (8) ensures that each flight has at most one pair of launch and recovery nodes. Constraint (9) states that if $m = n$, then $y_{dijmn} = 0$.

$$y_{d' i' j' m' n'} \left(\sum_{d \in D, d \neq d'} \sum_{(i',j') \in AL} \sum_{m \in V} \sum_{n \in V} y_{d' i' j' m' n'} + \sum_{d \in D, d \neq d'} \sum_{(j',i') \in AL} \sum_{m \in V} \sum_{n \in V} y_{d' j' i' m' n'} \right) = 0, \forall d' \in D, \forall (i',j') \in AL, \forall m', n' \in V \quad (10)$$

$$y_{d' i' j' m' n'} \left(\sum_{d \in D, d \neq d'} \sum_{(i',j') \in AL} \sum_{m \in V} \sum_{n \in V} y_{d' i' j' m' n'} + \sum_{d \in D, d \neq d'} \sum_{(j',i') \in AL} \sum_{m \in V} \sum_{n \in V} y_{d' j' i' m' n'} \right) = 0, \forall d' \in D, \forall (i',j') \in AL, \forall m', n' \in V \quad (11)$$

$$y_{d' i' j' m' n'} \left(\sum_{(i',j') \in AL, j' \neq i'} \sum_{m \in V} \sum_{n \in V} y_{d' i' j' m' n'} + \sum_{(i',j') \in AL, i' \neq j'} \sum_{m \in V} \sum_{n \in V} y_{d' i' j' m' n'} \right) = 0, \forall d' \in D, \forall (i',j') \in AL, \forall m', n' \in V \quad (12)$$

Constraints (10)-(12) ensure that each node in the GV route can at most be either the launch or recovery node of one flight of one drone.

$$\varphi_{im}^T + \varphi_{im}^R = x_{im}, \forall m \in V, \forall i \in K_{TR} \quad (13)$$

$$\varphi_{im}^T \left(\sum_{d \in D} \sum_{(i,j) \in AL} \sum_{n \in V} y_{dijmn} - 1 \right) = 0, \forall i \in K_T, \forall m \in V \quad (14)$$

$$\varphi_{jm}^R \left(\sum_{d \in D} \sum_{(i,j) \in AL} \sum_{n \in V} y_{dijmn} - 1 \right) = 0, \forall j \in K_R, \forall m \in V \quad (15)$$

Constraints (13)-(15) indicate that if node m is the i^{th} node the GV visit and is a launch or recovery node of a flight, then this flight must start or end at node m .

$$\sum_{d \in D} \sum_{n \in V} y_{dijnm} \leq M(1 - \varphi_{im}^T + \varphi_{jm}^R), \forall (i,j) \in AL, \forall m \in V \quad (16)$$

$$\sum_{d \in D} \sum_{n \in V} y_{dijmn} \leq M(\varphi_{im}^T + 1 - \varphi_{jm}^R), \forall (i,j) \in AL, \forall m \in V \quad (17)$$

$$\sum_{n \in V} y_{dijmn} - \sum_{k \in V} y_{dikm} \leq M(x_{im} + x_{jm}), \forall d \in D, \forall (i,j) \in AL, \forall m \in V \quad (18)$$

$$\sum_{n \in V} y_{dijmn} - \sum_{k \in V} y_{dikm} \geq -M(x_{im} + x_{jm}), \forall d \in D, \forall (i,j) \in AL, \forall m \in V \quad (19)$$

Constraints (16) and (17) ensure that the drone visit nowhere before the launch node or after the recovery node if the launch and recovery nodes are different. Constraints (18) and (19) ensure the flow balance of node m if it is neither the launch nor recovery nodes of a flight.

$$\mu_{dij} \left(\sum_{m \in V} \sum_{n \in V} y_{dijmn} - 1 \right) \geq 0, \forall d \in D, \forall (i,j) \in AL \quad (20)$$

$$(1 - \mu_{dij}) \sum_{m \in V} \sum_{n \in V} y_{dijmn} \leq 0, \forall d \in D, \forall (i,j) \in AL \quad (21)$$

$$\sum_{(j,i) \in AL} \mu_{dji} + \sum_{(i,k) \in AL} \mu_{dik} \leq 1, \forall d \in D, \forall i \in K_{TR} \quad (22)$$

$$\mu_{dij} + \mu_{di'j'} \leq 1, \forall d \in D, \forall (i,j) \in AL, \forall (i',j') \in AL, i < i' < j < j' \quad (23)$$

Constraints (20)-(22) are to introduce the parameters μ_{dij} which indicate whether (v_i, v_j) is a pair of launch and recovery nodes of a flight of drone $d \in D$. Constraint (23) ensures that for two adjacent flights of a drone, the next flight can only be opened after the earlier one has been closed.

$$\sigma_{dijm} \geq 0, \forall d \in D, \forall (i,j) \in AL, \forall m \in V \quad (24)$$

$$\sigma_{dijm} - \sigma_{dijn} + |V|y_{dijmn} \leq |V| - 1 + Mx_{jn}, \forall d \in D, \forall (i,j) \in AL, \forall m, n \in V \quad (25)$$

Constraints (24) and (25) inspired by the Miller-Tucker-Zemlin (MTZ) formulation [47] eliminate potential sub tour in each flight, where Mx_{jn} is to allow the drone to be launched and recovered at the same node.

$$t_i^L \geq t_i^A, \forall i \in K_T \quad (26)$$

$$t_{i+1}^A \geq t_i^A, \forall i \in K_T, i < |R| \quad (27)$$

$$t_{i+1}^L \geq t_i^L, \forall i \in K_T, i < |R| \quad (28)$$

Constraints (26)-(28) impose some simple limitations on the departure and arrival times of the GV at each node. Constraint (26) ensures that the departure time of the GV at each node is not less than the arrival time. Constraints (27) and (28) ensure that the arrival or departure time of the GV at each node is not larger than the next one.

$$\delta_i \geq a_{mn}^{GV} - M(2 - x_{im} - x_{(i+1)n}), \forall i \in K, i < |R|, \forall m, n \in V \quad (29)$$

$$\omega_{im}(2 - x_{im} - x_{(i+1)m}) = 0, \forall i \in K, i < |R|, \forall m \in V \quad (30)$$

$$(1 - \omega_{im})(1 - x_{im} - x_{(i+1)m}) \geq 0, \forall i \in K, i < |R|, \forall m \in V \quad (31)$$

$$t_{i+1}^A \geq t_i^L + \frac{\delta_i}{v_{GV}} - M \sum_{m \in V} \omega_{im}, \forall i \in K_T, i < |R| \quad (32)$$

Constraint (29) introduces the parameters δ_i which indicate the distance for the GV from v_i to v_{i+1} . Constraints (30) and (31) introduce the parameters ω_{im} which indicate whether the v_i and v_{i+1} are both node m . Constraint (32) indicates that the arrival time of the GV at each node is not less than the departure time at the former node plus the time it takes the GV to travel.

Algorithm 1

Framework of RVND-SA.

```

1      Input:  $T_0, T_{min}, \gamma, s_0, NL$ 
2      Output:  $s^{BEST}$ 
3      Initialize  $s^{BEST} \leftarrow s_0, s \leftarrow s_0, T \leftarrow T_0$ 
4      While  $T > T_{min}$  do
5           $s' \leftarrow RVND(NL, s)$ 
6           $\Delta f \leftarrow (f(s') - f(s)) / f(s)$ 
7          If  $\Delta f < 0$  then
8               $s \leftarrow s'$ 
9          Else
10             Accept  $s \leftarrow s'$  with a probability  $\exp(-\Delta f / T)$ 
11          End if
12          If  $f(s) < f(s^{BEST})$  then
13               $s^{BEST} \leftarrow s$ 
14          End if
15           $T = \gamma T$ 
16      End while

```

Algorithm 2

Randomized Variable Neighborhood Descent (RVND).

```

1      Input:  $NL, s_0$ 
2      Output:  $s'$ 
3      Initialize  $s' \leftarrow s_0, NL$ 
4      Shuffle  $NL$ 
5      While  $NL \neq \emptyset$  do
6          Choose the first neighborhood in  $NL$ , denoted as  $NL^{(k)}$ 
7          Find the best neighboring solution  $s'' \in NL^{(k)}(s')$ 
8          If  $f(s'') < f(s')$  then
9               $s' \leftarrow s''$ 
10             Reinitialize and shuffle  $NL$ 
11          Else
12             Remove  $NL^{(k)}$  from  $NL$ 
13          End if
14      End while

```

$$t_j^D \geq t_i^A + \sum_{m \in V} \sum_{n \in V} y_{dijmn} c_{mn}^U / v^U, \forall d \in D, \forall (i, j) \in AL \quad (33)$$

$$t_j^D \geq t_{i-1}^D + \sum_{m \in V} \sum_{n \in V} y_{dijmn} c_{mn}^U / v^U - M \sum_{m \in V} \omega_{i-1,m} - M \left(2 - \sum_{(k,i-1) \in AL} \mu_{d,k,i-1} - \mu_{dij} \right), \forall d \in D, \forall (i, j) \in AL \quad (34)$$

Constraint (33)-(34) indicates that the landing time of drone d is not less than the launch time plus the travel time of this flight, while the launch time is related to t_i^A and t_{i-1}^D . Constraint (33) means that the launch time of drone d won't be later than t_i^A in all cases. Nevertheless, in some cases that the drone d lands on v_{i-1} and then launches again from v_i , while the nodes indicated by v_{i-1} and v_i are actually the same, it couldn't launch before it has landed (t_{i-1}^D) as indicated by constraint (34).

$$t_j^L \geq t_j^D, \forall j \in K_R \quad (35)$$

$$\mu_{dij} (t_j^A - t_i^A - P) \leq 0, \forall d \in D, \forall (i, j) \in AL \quad (36)$$

Constraint (35) indicates that the GV won't leave until it recovers the drone that is supposed to land there. Constraint (36) indicates that the time cost of each flight cannot exceed the maximum flying time P .

4. The RVND-SA algorithm

This section describes the proposed method for solving GVD-ARP, named randomized variable neighborhood descent algorithm combined with simulated annealing algorithm (RVND-SA). The overall algorithm framework, initial solution method, and neighborhood search strategies of RVND-SA are elaborated respectively in the following subsections.

4.1. Framework of RVND-SA

The variable neighborhood descent (VND) [48] has been applied to solve NP-hard problems such as VRP [49]. The most important feature of randomized VND (RVND) is to shuffle the order of the neighborhood list before each iteration, instead of using the original fixed order as in variable neighborhood descent (VND), which increases the search stochasticity of the algorithm. In order to improve the efficiency of the algorithm, we adopt the Metropolis criterion to accept degenerate solutions with a certain probability so that the algorithm can escape from the local optimum and avoid premature convergence. Meanwhile, four



Fig. 4. Schematic diagram of a drone flight $19, \{(23, 22), (17, 16)\}, 5$.

problem-specific neighborhood search strategies are designed for finding high-quality solutions. Algorithm 1 presents the framework of RVND-SA.

RVND-SA first inputs the initial temperature T_0 , end temperature T_{min} , cooling rate γ , initial solution s_0 , and neighborhood list NL . The initial solution s_0 is obtained by the four steps described in Section 4.2. The best solution s^{BEST} , current solution s , and current temperature T are initialized in line 3. Then, in the main loop of RVND-SA (lines 4–16), the solution is iteratively updated until a termination criterion is met. First, the function $RVND(NL, s_0)$ is called and a new solution s' is obtained, which is detailed in Algorithm 2. The number of searches for each neighborhood is n_{ND} . The current solution s will be updated if it is better than the new solution s' (lines 7–8). When the current solution s is not better than s' , accept a worse solution s' with a probability of $\exp(-\Delta f / T)$ according to the Metropolis criterion (lines 10–12). Finally, update s to the best solution s^{BEST} if s is better than s^{BEST} . Furthermore, each time after one main loop is finished, T is updated accordingly (lines 15).

The pseudocode of the RVND is presented in Algorithm 2. First, the new solution s' , and a neighborhood list (NL) which contains a certain number of neighborhood structures are initialized (line 3). Second, the visit order of the neighborhood structures is shuffled in line 4. Then, in the main loop (lines 5–14), the first neighborhood in NL denoted as $NL^{(k)}$ is chosen (line 6) and then the best neighboring solution s'' is determined (line 7). If current solution s'' is better than the new solution s' , s' is updated and the NL is reinitialized and shuffled (lines 8–10). Otherwise, $NL^{(k)}$ is removed from NL (line 12).

4.2. Initial solution generation method

Due to the battery endurance, a drone can only visit a limited number of arcs in a single flight. To describe the route of a drone flight clearly, let $SATE_i$ denotes a set of the i^{th} successively visited target arcs in a drone flight. As shown in Fig. 4, a drone is launched from node 19 and recovered at node 5 after visiting target arcs (23, 22) and (17, 16) included in $SATE_i = \{(23, 22), (17, 16)\}$. The route of a drone flight that visits $SATE_i$ can be expressed by $s, SATE_i, n$, where the drone is launched from s and recovered at n after serving the target arcs in set $SATE_i$.

The initial solution s_0 can be generated by the initial solution generation method, which consists of four steps as shown in Fig. 5.

First, the direction of a target arc a_{ij} is determined by random gen-

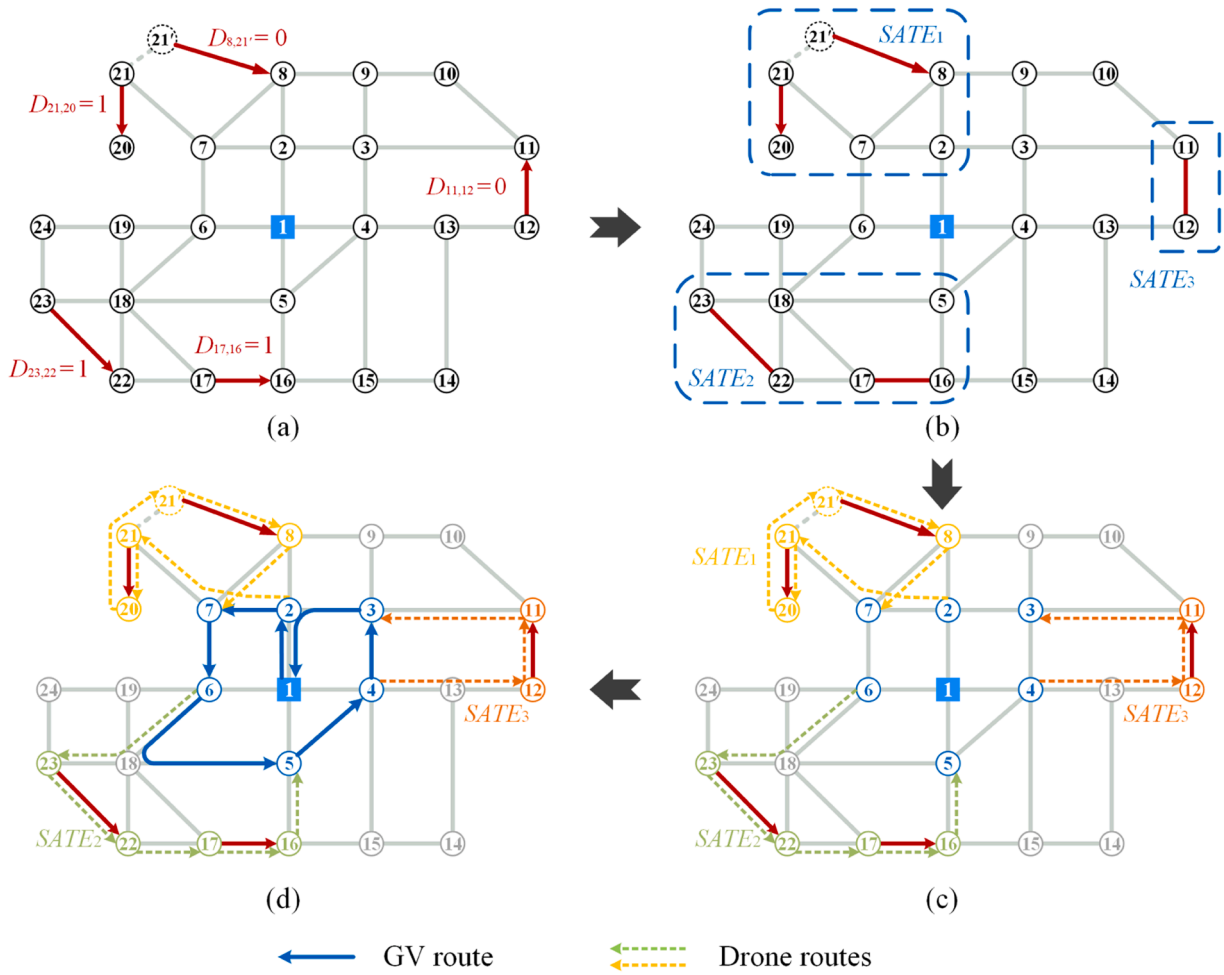


Fig. 5. Schematic diagram of initial solution generation.

eration and recorded in a binary variable D_{ij} . Wherein, $D_{ij} = 1$ means that the patrol direction of the target arc is consistent with that recorded in TA , while $D_{ij} = 0$ means the direction is opposite. For example, in Fig. 5(a), the set of target arcs is recorded as $TA = \{(8, 21'), (21, 20), (23, 22), (17, 16), (11, 12)\}$. The patrol direction of the target arc $a_{11,12}$ is marked by $D_{11,12} = 0$, indicating that the visiting direction for the target arc is $12 \rightarrow 11$.

Second, the target arcs are divided into multiple groups by the following steps. All target arcs are randomly ordered, taking $\{(8, 21'), (21, 20), (23, 22), (17, 16), (11, 12)\}$ as an example, and then as many target arcs as possible are assigned to a flight from front to back. For each flight, the total time required for the drone to complete patrol tasks in the determined order cannot exceed the maximum flight time. As shown in Fig. 5(b), the five target arcs are divided into three sets $SATE_1 = \{(8, 21'), (21, 20)\}$, $SATE_2 = \{(23, 22), (17, 16)\}$, and $SATE_3 = \{(11, 12)\}$.

Third, the sub-route to visit the target arcs in a set $SATE_i$ is determined by connecting all target arcs from end to end. For example, we form the sub-route that visits $SATE_2$ through $23-22-17-16$. Then, a set $AL_{SATE_i} = \{s_1, n_1, s_2, n_2, \dots\}$ is defined to store all possible pairs of launch and recovery nodes s and n in a drone flight which visit the target arcs in set $SATE_i$. For a pair of launch and recovery nodes s, n , the time of the flight which starts from node s and ends at node n cannot exceed the maximum flight time P . For example, if the time cost of a drone route 6, $SATE_2$, 4 exceeds P , the launch and recovery nodes 6, 4 cannot be in set AL_{SATE_2} . Randomly select a pair of launch and recovery nodes in AL_{SATE_i} as the launch and recovery nodes of $SATE_i$. As Fig. 5(c) displayed, the

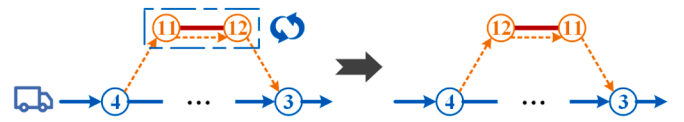


Fig. 6. Switch the direction of a single target arc.

pairs of launch and recovery nodes for $SATE_1$, $SATE_2$ and $SATE_3$ are 2, 7, 6, 5, and 4, 3, respectively. The routes of all drone flights can be expressed by 2, $SATE_1$, 7, 6, $SATE_2$, 5, and 4, $SATE_3$, 3.

Through the above three steps, the route of each drone flight can be obtained. Finally, randomly order the visiting sequence of all drone flights, and connect the launch and recovery nodes of above drone flights according to the sequence. Fig. 5(d) illustrates that the GV starts from node 1, and then passes the launch and recovery nodes of the drone flights 2, $SATE_1$, 7, 6, $SATE_2$, 5, and 4, $SATE_3$, 3 in sequence. Thus, the GV route can be expressed in terms of $R = \{1, 2, 7, 6, 5, 4, 3, 1\}$, where only the patrol center or other crucial nodes involved in launching or recovering drones are recorded. The GV travels along the shortest path between two adjacent nodes.

4.3. Neighborhood search strategies

Four problem-specific neighborhood strategies are designed to improve the initial solution, which will be explained with schematic diagrams in this subsection.

From Fig. 6 to Fig. 12, the GV route is represented by blue solid lines,

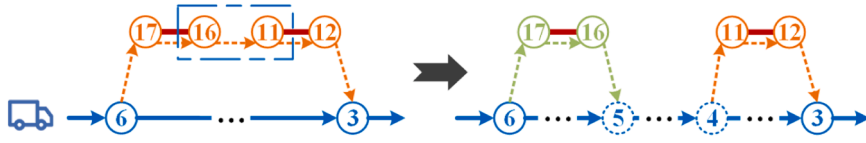


Fig. 7. Switch the connection state of multiple task arcs.

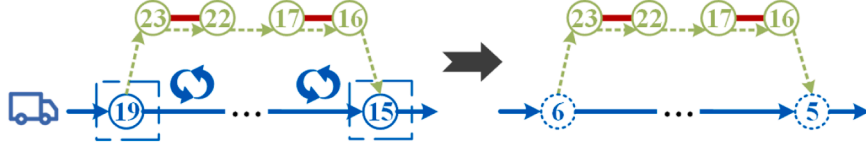


Fig. 8. Change start-end nodes.

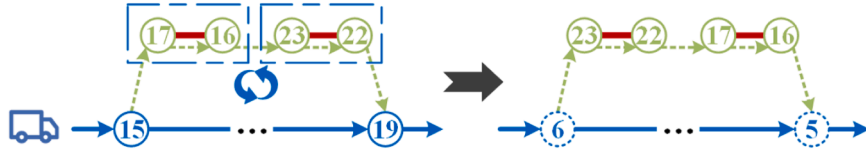


Fig. 9. Exchange the Visiting Order in a Drone Flight.

routes of drone flights are represented by dashed lines, and the forward direction is represented by the arrow. The lines represent the shortest distance between the two nodes they connect in the road network, rather than the actual arcs, which can be obtained by Dijkstra algorithm [50].

4.3.1. Switch

This is a problem-based neighborhood strategy with two different operators. The first is to switch the visiting direction D_{ij} of a single visited target arc, and the second is to switch the connection state of multiple target arcs. In this strategy, one target arc a_{ij} is selected, and different switch operators are performed according to whether other target arcs are visited in the same flight. When there are no other target arcs in the same flight that visits a_{ij} , the first operator is performed. Otherwise, perform the first or second operator randomly.

- Switch the direction of a single target arc. In this operator, exchange the two nodes of a_{ij} along with the launch and recovery nodes accordingly. As shown in Fig. 6, the route of the drone flight is changed from 4 to 11–12–3 to 4–12–11–3 by this operator.
- Switch the connection state of multiple task arcs. Separate a target arc a_{ij} from the original flight and generate a new drone flight to visit a_{ij} according to the first three steps in Section 4.2. In the original flight, the number of target arcs is reduced by one and the launch and recovery nodes are reselected from the new set AL_{SATE_i} . In Fig. 7, the launch and recovery nodes are changed from 6, 3 to 4, 3, and a new drone route 6–17–16–5 which visits $a_{17,16}$ is generated.

4.3.2. Change start-end nodes

In the initial solution constructed by the four steps in Section 4.2, the launch and recovery nodes of a flight that visits $SATE_i$ are randomly selected in the AL_{SATE_i} . Thus, changing the launch and recovery nodes of a drone flight may improve the solution. In this strategy, reselect a pair of launch and recovery nodes in AL_{SATE_i} as the new launch and recovery nodes of the drone flight that visits a_{ij} . The operator is illustrated in Fig. 8.

4.3.3. Change flight sequence

As the launch and recovery sequences of different drone flights are randomly determined in the initial solution, changing the flight se-

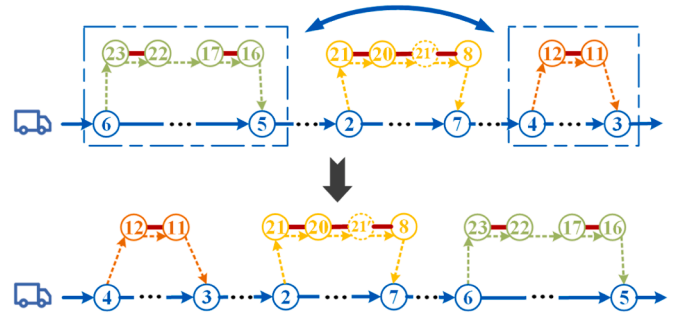


Fig. 10. Exchange the Sequence of Two Drone Flights.

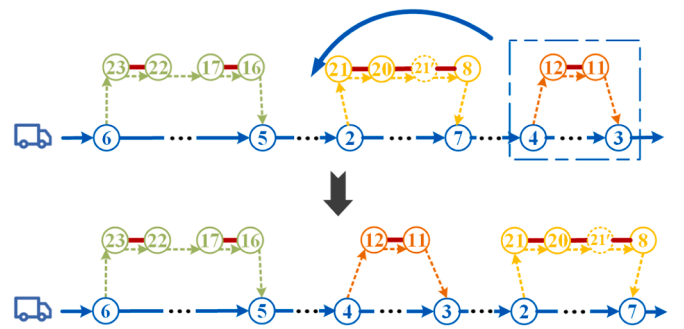


Fig. 11. Deletion-Reinsertion of a Drone Flight.

quences may also improve the solution. The strategy Change Flight Sequence contains three operators.

- Exchange the visit order in a drone flight. In a drone flight that visits no less than two target arcs, taking $SATE_2$ as an example, exchange the visit order for the two randomly selected target arcs, and reselect the launch and recovery nodes accordingly. As presented in Fig. 9, the visiting order for $a_{17,16}$ and $a_{23,22}$ are exchanged, so that $SATE_2$ changes from $\{(17, 16), (23, 22)\}$ to $\{(23, 22), (17, 16)\}$, and AL_{SATE_2} is

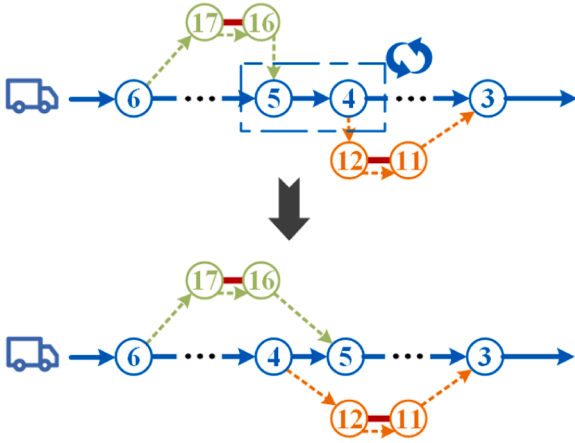


Fig. 12. A Delay Recovery Sequence operator.

updated accordingly. Then the launch and recovery nodes of $SATE_2$ are re-chosen in the new AL_{SATE_2} .

- Exchange the sequence of two drone flights. Fig. 10 illustrates that the visit order of the two flights that visits $\{(23, 22), (17, 16)\}$ and $\{(12, 11)\}$ are exchanged. Meanwhile, the launch and recovery nodes of these two flights remain the same.
- Deletion-reinsertion of a drone flight. Delete a drone flight from the GV route and then reinsert it into another position. In Fig. 11, the drone flight 4–12–11–3 is deleted from its original position and reinserted before node 2 in the GV route.

4.3.4. Delay recovery sequence

As introduced in Section 4.2, a drone flight in the initial solution is usually short, so it may happen that a drone is waiting for the GV at the recovery node. It is difficult to ensure that the waiting time of a drone at the recovery node is minimized due to the randomly selected launch and recovery nodes. If the GV continues to drive while waiting for the drone to be recovered and launches other drones, it is very likely to improve the solution. In this strategy, the recovery node of a drone flight s , $SATE_i$, n is randomly moved to a position after the launch or recovery node of another flight after s , $SATE_i$, n in the GV route. As presented in Fig. 12, the recovery node 5 of the former flight 6–17–16–5 is moved after node 4, which is the launch node of the next drone flight 4–12–11–3. At the same time, the GV route is changed to 6–4–5–3.

5. Performance evaluation

In this section, a set of experiments have been conducted. We compare the solution obtained from RVND-SA and other heuristics with the optimal solution from the Gurobi solver on small-scale instances to verify the mathematical model. Further, RVND-SA is compared with GA, VND, VND-SA, and RVND [51] on 35 instances of different sizes to test its efficiency. The sensitivity analyses are conducted on two major parameters of drones. In addition, a real-world case is applied to further verify the applicability of the proposed model and method.

5.1. Experiment design

Since GVD-ARP with multiple drones and a GV investigated in this study is a new problem, there are no publicly benchmark instances and algorithms. In this paper, the proposed RVND-SA is compared with four metaheuristic algorithms, including the Memetic algorithm (MA), VND [52], the VND with the simulated annealing algorithm (VND-SA), and the randomized variable neighborhood descent search (RVND) [51].

The memetic algorithm is an evolutionary algorithm that combines global and local search [53], of which the framework for solving the GVD-ARP in this work is inspired by [54]. The number of individuals in

Table 3

The size of the small-scale instances.

Network	$ V $	$ A $	$ TA $	Drones	Instance
T1	6	12	1	1	T1-1-1
T2	7	16	1	1	T2-1-1
			2	2	T2-2-2
			3	2	T2-3-2
T3	8	23	1	1	T3-1-1
			2	2	T3-2-2
			3	2	T3-3-2
T4	9	28	3	2	T4-3-2
			3	3	T4-3-3

the population is represented by N_{MA} . After generating the initial population by the method described in Section 4.2, the MA enters an evolutionary process. In each generation, every two individuals are randomly paired to perform crossover by swapping the directions of the target arc so that two offspring have been generated, and then the MA tries to improve them by local search. VND is proposed by Mladenović and Hansen [55] first. The SA framework is combined with the VND to form the VND-SA. The RVND is an improved version of VND, wherein the order of neighborhoods is shuffled at each iteration to enhance the algorithm's global search capability. For fair comparison, all the comparative algorithms use the same neighborhood search strategies as RVND-SA.

In order to verify the correctness of the model, some experiments should be carried out based on the model itself and using an exact algorithm for small-scale instances to find the optimal solutions. We manually generated four network scenarios T1, T2, T3, and T4. For each network scenario, a different number of target arcs are generated, and accordingly, the number of drones carried by the GV is also different. The name of each small-scale instance consists of the network, the number of tasks, and the number of drones. The instances are described in Table 3, where $|V|$, $|A|$, and $|TA|$ stands for the number of nodes, arcs, and target arcs, respectively.

Since the routing problem for traffic patrol by coordinating a ground vehicle and drones is an emerging new problem, there is no standard test instance for the GVD-ARP. To generate scenarios, we extract several regions from five transportation networks: SiouxFalls, GoldCoast, Chicago-regional, Philadelphia, and Sydney. All network data can be accessed in the GitHub repository named Transportation Networks [56]. Except that SiouxFalls is of moderate size, the networks are too large and contain too many nodes to be applied to test the performance of algorithms without the following processing. The horizontal and vertical coordinate ranges of the segments captured in the network are represented by $[x_{min}, x_{max}]$ and $[y_{min}, y_{max}]$, respectively. Then the nodes not connected to the majority due to this cutting will be deleted. In addition, the length of each link between nodes is expanded by *ratio* due to the difference in the scale of maps. The characteristics of generated instances based on the transportation networks are presented in Table 4.

The parameters related to the ground vehicle and the drones are set as shown in Table 5. Note that the number of drones is 3 in all transportation-network-based instances, while in the small-scale instances, it varies based on Table 3.

5.2. Algorithm parameter tuning

The optimal values of other parameters may vary with the size of the instances, so extensive experiments are required. We grouped the generated 35 instances into three scales according to the number of nodes ranging from 24 to 110, 150 to 210, and 240 to 310, respectively. In each scale, CR050-04, CR153-12, and CR253-20 are selected to test the parameters of all algorithms through grid search. In terms of algorithms with simulated annealing procedures, we referred to the parameter settings of Ropke and Pisinger [57] that the initial temperature is set to a value that a solution with an objective up to 5% worse

Table 4

The characteristics of the transportation-network-based instances.

Network	ratio	$ V $	$ E $	$[x_{\min}, x_{\max}]$	$[y_{\min}, y_{\max}]$	$ TA $	Instance
SiouxFalls	1	24	76	$[-96.7934, -96.6934]$	$[43.4907, 43.6128]$	2	SF024-02
						3	SF024-03
						4	SF024-04
						5	SF024-05
						6	SF024-06
						7	SF024-07
						8	SF024-08
						30	SF024-30
GoldCoast	14	48	120	$[153.4022, 153.4105]$	$[-27.9549, -27.9410]$	4	GC048-04
		101	248	$[153.4005, 153.4123]$	$[-27.9626, -27.9410]$	8	GC101-08
		153	386	$[153.4003, 153.4162]$	$[-27.9674, -27.9406]$	12	GC153-12
		199	511	$[153.3981, 153.4179]$	$[-27.9699, -27.9405]$	16	GC199-16
		249	616	$[153.4191, 153.4408]$	$[-28.0577, -28.0012]$	20	GC249-20
		300	743	$[153.4151, 153.4413]$	$[-28.0596, -28.0012]$	24	GC300-24
		30	GC300-30				
		30	GC300-30				
Chicago-regional	2	50	144	$[568,600, 582,000]$	$[1,957,500, 1,984,400]$	4	CR050-04
		101	306	$[560,100, 582,000]$	$[1,930,800, 1,984,400]$	8	CR101-08
		153	463	$[555,800, 582,000]$	$[1,914,775, 1,984,400]$	12	CR153-12
		205	628	$[551,000, 582,000]$	$[1,907,000, 1,984,400]$	16	CR205-16
		253	786	$[541,600, 582,000]$	$[1,907,000, 1,984,400]$	20	CR253-20
		307	955	$[520,300, 582,000]$	$[1,910,000, 1,984,400]$	24	CR307-24
		30	CR307-30				
		30	CR307-30				
Philadelphia	2	50	140	$[29,469, 29,681]$	$[76,711, 76,886]$	4	PA050-04
		103	302	$[29,352, 29,837]$	$[76,700, 77,098]$	8	PA103-08
		151	444	$[29,352, 30,100]$	$[76,700, 77,126]$	12	PA151-12
		207	616	$[29,350, 30,100]$	$[76,700, 77,467]$	16	PA207-16
		254	774	$[29,422, 30,100]$	$[76,404, 77,467]$	20	PA254-20
		305	932	$[29,305, 30,100]$	$[76,404, 77,500]$	24	PA305-24
		30	PA305-30				
		30	PA305-30				
Sydney	5	52	118	$[151.080, 151.093]$	$[-33.797, -33.787]$	4	SD052-04
		101	238	$[151.080, 151.095]$	$[-33.808, -33.787]$	8	SD101-08
		151	360	$[151.071, 151.097]$	$[-33.809, -33.786]$	12	SD151-12
		208	480	$[151.076, 151.126]$	$[-33.805, -33.786]$	16	SD208-16
		254	590	$[151.071, 151.12]$	$[-33.809, -33.786]$	20	SD254-20
		295	680	$[151.072, 151.127]$	$[-33.810, -33.786]$	24	SD295-24
		30	SD295-30				
		30	SD295-30				

Table 5

Values of involved parameters.

Parameters	Value (unit)
Number of ground vehicles	1 (vehicle)
Number of drones carried by the ground vehicle	3 (drones)
v^{veh} (Speed of the vehicle)	30 (km/h)
v^d (Speed of the drones)	35 (km/h)
P (Limited time of a drone flight)	0.67 (hour)

than the current solution is accepted with probability 0.5, and the end temperature in the last iteration is 0.2 percent of the initial. The configuration of other parameters for all algorithms in each scale is finally determined, as shown in Table A.2 (in the appendix).

5.3. Experiment result

The mathematical model was coded using the Gurobi optimizer and

implemented with Python 3.6. RVND-SA and other metaheuristics were programmed in MATLAB R2016a. All programs run on an x64 Windows 10 desktop computer with a 2.90 GHz i7-10700F CPU and 32.0 GB memory.

5.3.1. Comparison of the proposed mathematical model and the metaheuristics

The proposed mathematical model was implemented for nine small-scale instances to find the optimal solutions. The metaheuristics were also conducted to solve the same instances and compare their results with the optimal solution. The performance measures considered in the comparison are (i) the best objective found by the algorithm (Best): the total time (in hours) it takes for the drones to complete all petrol tasks and return to the center with the GV, (ii) the average CPU time (in seconds) for the algorithm (Time), (iii) percentage gap (Gap): $\text{Gap} = 100[(z_P - z_D) / z_P]$ where z_P is the incumbent objective value, and z_D is the lower bound. All metaheuristics are set to run 30 times, and the total time the Gurobi optimizer expended is limited to 3 h.

Table 6

Result of the mathematical model and metaheuristics on small-scale instances.

No.	Instance	Gurobi Best	Time	Gap	MA Best	Time	VND Best	Time	VND-SA Best	Time	RVND Best	Time	RVND-SA Best	Time
1	T1-1-1	0.40	0.01	0.0	0.40	36.08	0.40	24.46	0.40	30.90	0.40	16.42	0.40	19.31
2	T2-1-1	0.51	0.04	0.0	0.51	37.54	0.51	26.21	0.51	33.31	0.51	17.24	0.51	20.51
3	T2-2-2	0.63	0.05	0.0	0.63	36.84	0.63	25.62	0.63	31.97	0.63	17.16	0.63	20.32
4	T2-3-2	0.83	2.64	0.0	0.84	41.14	0.83	26.56	0.94	32.45	0.83	18.85	0.83	21.42
5	T3-1-1	0.82	13.49	0.0	0.82	43.65	1.23	28.86	0.82	35.82	0.82	18.90	0.82	21.56
6	T3-2-2	0.86	1150.39	0.0	0.86	41.41	0.86	26.87	0.86	30.81	0.86	18.84	0.86	21.47
7	T3-3-2	1.02	2722.89	0.0	1.16	42.62	1.23	29.86	1.17	36.09	1.17	21.66	1.02	23.79
8	T4-3-2	0.73	3720.12	0.0	0.73	43.08	0.73	28.39	0.74	35.66	0.73	19.57	0.73	22.98
9	T4-3-3	0.63	10,800.19	57.2	0.63	42.21	0.63	27.79	0.63	34.96	0.63	19.72	0.63	22.12

Table 7
Performance of RVND-SA algorithm and the compared algorithms.

No.	Instance	MA Best	Average	Time	VND Best	Average	Time	VND-SA Best	Average	Time	RVND Best	Average	Time	RVND-SA Best	Average	Time
1	SF024-02	1.729	1.729	41.13	1.729	1.729	37.53	1.729	1.729	33.07	1.729	1.729	41.35	1.729	1.729	29.10
2	SF024-03	1.590	1.712	43.35	1.590	1.818	31.45	1.590	1.762	36.52	1.590	1.736	44.15	1.590	1.738	28.78
3	SF024-04	1.700	1.859	44.62	1.729	2.061	32.48	1.729	1.910	39.10	1.738	1.966	42.76	1.629	1.829	29.14
4	SF024-05	1.829	2.274	47.98	2.086	2.466	31.53	1.924	2.410	39.48	1.895	2.391	47.49	1.767	2.265	32.31
5	SF024-06	2.152	2.594	49.78	2.252	2.966	32.25	2.386	2.821	36.90	2.052	2.609	47.96	1.995	2.387	32.25
6	SF024-07	2.224	2.923	49.79	2.452	3.216	33.87	2.510	3.017	35.64	2.357	2.987	49.32	2.152	2.703	33.46
7	SF024-08	2.400	3.285	43.81	2.781	3.447	33.34	2.695	3.397	38.22	2.148	3.098	43.27	2.429	2.899	28.83
8	GC048-04	1.229	1.383	44.57	1.185	1.412	38.68	1.229	1.444	44.67	1.198	1.393	41.25	1.207	1.323	34.88
9	GC101-08	2.631	3.391	53.75	2.841	3.247	32.69	2.757	3.419	35.79	2.610	3.245	50.78	2.519	3.017	40.95
10	GC153-12	4.401	5.505	63.32	3.858	4.653	45.50	4.299	5.147	46.42	3.922	4.692	64.25	3.796	4.282	43.15
11	GC199-16	7.613	8.865	66.92	6.121	7.430	43.25	6.510	7.463	50.71	6.201	7.082	69.12	5.520	6.711	44.86
12	GC249-20	11.592	15.130	97.49	10.585	12.203	112.81	10.719	12.678	100.24	10.183	11.981	96.41	9.622	10.593	81.35
13	GC300-24	16.723	21.816	102.38	13.118	16.432	127.85	15.198	18.215	96.56	11.823	14.858	90.74	11.590	13.366	75.30
14	GC300-30	20.472	26.171	111.15	16.333	20.662	104.12	17.351	21.458	103.67	14.909	18.984	102.66	13.922	17.022	78.99
15	CR050-04	0.389	0.444	55.80	0.362	0.435	39.49	0.349	0.413	52.56	0.350	0.421	48.34	0.343	0.393	38.60
16	CR101-08	1.441	1.902	56.93	1.251	1.768	38.84	1.388	1.799	44.83	1.340	1.742	51.85	1.234	1.623	41.85
17	CR153-12	2.847	3.335	57.15	2.427	3.015	49.38	2.665	2.949	60.42	2.533	2.979	62.57	2.292	2.816	45.20
18	CR205-16	5.129	6.723	54.29	4.488	5.647	39.23	4.672	5.965	43.39	3.888	5.154	58.25	3.332	4.755	39.58
19	CR253-20	6.831	9.019	106.72	5.460	6.785	139.15	5.956	6.921	117.81	5.398	6.488	106.89	5.374	6.194	97.05
20	CR307-24	9.339	12.975	113.59	7.918	9.914	172.95	8.296	9.655	99.17	7.581	9.136	96.50	5.409	7.587	99.90
21	CR307-30	13.263	17.661	128.68	10.652	13.008	139.26	11.694	13.565	122.78	9.129	11.793	119.70	8.229	10.718	94.82
22	PA050-04	0.208	0.265	43.12	0.201	0.261	38.56	0.205	0.260	47.09	0.181	0.261	40.96	0.204	0.246	33.87
23	PA103-08	0.749	0.997	40.34	0.764	0.969	35.19	0.674	0.959	42.41	0.668	0.979	38.19	0.653	0.871	34.24
24	PA151-12	1.868	2.505	63.55	1.518	2.095	51.47	1.454	2.128	67.58	1.550	2.115	61.50	1.453	1.944	41.43
25	PA207-16	3.891	4.774	76.90	2.766	3.814	51.00	2.821	3.811	61.33	2.976	3.837	73.47	2.752	3.505	50.06
26	PA254-20	5.222	6.376	151.66	3.342	4.769	162.94	4.043	4.872	180.36	4.131	4.926	144.52	3.034	4.475	132.07
27	PA305-24	6.666	8.446	288.71	5.511	6.142	186.69	5.298	6.259	192.77	5.108	6.367	220.12	4.713	5.646	179.99
28	PA305-30	7.895	11.616	294.63	7.499	8.734	228.70	7.278	8.523	217.75	6.778	8.587	195.78	7.039	8.020	201.65
29	SD052-04	0.361	0.395	35.77	0.348	0.376	28.00	0.348	0.388	37.74	0.350	0.383	33.60	0.348	0.365	27.06
30	SD101-08	1.042	1.443	45.20	1.036	1.399	29.37	0.923	1.324	38.87	1.014	1.287	42.41	0.944	1.196	34.27
31	SD151-12	1.710	2.637	70.33	1.679	2.201	51.67	1.705	2.230	68.68	1.496	2.191	69.65	1.354	2.011	47.86
32	SD208-16	3.275	4.568	82.08	2.648	3.377	52.84	2.648	3.497	66.54	2.528	3.343	78.53	2.413	3.171	53.60
33	SD254-20	4.620	6.354	134.02	3.448	4.348	169.81	3.124	4.211	166.49	3.411	4.620	139.76	2.843	3.992	130.01
34	SD295-24	4.968	8.394	155.05	5.175	6.276	187.33	4.825	5.928	159.07	4.475	6.164	137.86	4.132	5.496	130.59
35	SD295-30	7.333	11.760	170.13	6.782	8.156	186.46	7.068	7.773	137.91	6.390	7.892	238.07	6.102	7.453	121.88

Table 8The p and Gap of the algorithms on the dataset.

No.	Instance	MA		VND		VND-SA		RVND	
		p	Gap	p	Gap	p	Gap	p	Gap
1	SF024-02	1.000	0.00	0.605	0.00	0.605	0.00	0.781	0.00
2	SF024-03	0.779	0.00	0.024	0.00	0.165	0.00	0.285	0.00
3	SF024-04	0.032	4.39	0.000	6.14	0.036	6.14	0.000	6.73
4	SF024-05	0.427	3.50	0.001	18.06	0.006	8.89	0.035	7.28
5	SF024-06	0.003	7.88	0.000	12.89	0.000	19.57	0.007	2.86
6	SF024-07	0.051	3.32	0.000	13.94	0.003	16.59	0.003	9.51
7	SF024-08	0.000	-1.18	0.000	14.51	0.000	10.98	0.006	-11.57
8	GC048-04	0.022	1.82	0.020	-1.82	0.027	1.77	0.018	-0.77
9	GC101-08	0.000	4.47	0.011	12.81	0.000	9.45	0.002	3.63
10	GC153-12	0.000	15.93	0.001	1.63	0.000	13.24	0.000	3.32
11	GC199-16	0.000	37.91	0.000	10.88	0.000	17.93	0.019	12.33
12	GC249-20	0.000	20.47	0.000	10.01	0.000	11.40	0.000	5.83
13	GC300-24	0.000	44.29	0.000	13.18	0.000	31.13	0.001	2.01
14	GC300-30	0.000	47.05	0.000	17.32	0.000	24.63	0.001	7.09
15	CR050-04	0.000	13.28	0.000	5.32	0.052	1.69	0.004	2.02
16	CR101-08	0.000	16.77	0.004	1.41	0.001	12.50	0.015	8.64
17	CR153-12	0.000	24.22	0.003	5.91	0.027	16.29	0.037	10.51
18	CR205-16	0.000	53.94	0.000	34.72	0.000	40.24	0.015	16.69
19	CR253-20	0.000	27.10	0.001	1.59	0.000	10.83	0.042	0.44
20	CR307-24	0.000	72.66	0.000	46.38	0.000	53.38	0.000	40.15
21	CR307-30	0.000	61.17	0.000	29.43	0.000	42.10	0.000	10.93
22	PA050-04	0.014	2.25	0.060	-1.17	0.050	0.56	0.035	-11.32
23	PA103-08	0.002	14.62	0.006	16.91	0.015	3.08	0.002	2.29
24	PA151-12	0.000	28.59	0.037	4.47	0.014	0.12	0.012	6.67
25	PA207-16	0.000	41.42	0.001	0.53	0.001	2.53	0.006	8.15
26	PA254-20	0.000	72.09	0.018	10.13	0.008	33.22	0.003	36.15
27	PA305-24	0.000	41.46	0.000	16.94	0.000	12.42	0.000	8.38
28	PA305-30	0.000	12.16	0.001	6.53	0.002	3.38	0.014	-3.71
29	SD052-04	0.000	3.55	0.012	0.00	0.012	0.00	0.003	0.51
30	SD101-08	0.000	10.39	0.004	9.72	0.048	-2.28	0.020	7.40
31	SD151-12	0.000	26.29	0.005	23.97	0.001	25.91	0.003	10.45
32	SD208-16	0.000	35.71	0.016	9.70	0.000	9.74	0.045	4.75
33	SD254-20	0.000	62.52	0.004	21.29	0.037	9.89	0.000	19.99
34	SD295-24	0.000	20.24	0.000	25.24	0.001	16.77	0.000	8.30
35	SD295-30	0.000	20.18	0.016	11.15	0.007	15.83	0.003	4.72

Table 6 reports the results of the mathematical model and the metaheuristics. For each instance, the best objective is expressed in **bold**. It is observed that Gurobi finds the optimal solution in most instances, except that in instance T4-3-3 only the objective with a Gap of 57.2 was found. The experiment shows that the exact algorithm takes very little CPU time in some instances, but it has a significant growth trend with the increase of the instance size. The metaheuristic algorithms have acceptable running times for every small-scale instance and have found the optimal solution for most, while only RVND-SA obtained as good results as the exact algorithm on all small-scale instances. Thus, these results verify the correctness of both the model and algorithms and the importance of metaheuristics in solving large-scale instances.

5.3.2. Comparison of RVND-SA with peer algorithms on the transportation-network-based instances

To compare the overall performance of RVND-SA with the other four peer algorithms, we run each algorithm 30 times. Table 7 shows the performance of RVND-SA and its competitors on 35 generated instances. The best minimum and average objectives are expressed in **bold** for each instance.

It can be seen in Table 7 that the best objective obtained by RVND-SA is better than that of MA, VND, VND-SA, and RVND in 30 of 35 instances, and the average objective it is 34 of 35. As the number of target arcs increases, the advantage of the RVND-SA becomes more obvious. On the instances with smaller scales, there is little difference in the time-consuming of VND, VND-SA, RVND, and RVND-SA as four variants of Neighborhood Search. However, RVND and RVND-SA take a bit longer on large-scale instances than VND and VND-SA, especially on instances with no less than 20 target arcs. It can be explained that the first two need to visit different neighborhoods, and it searches for a larger solution space when solving the problem, which requires more computation

resources. The memetic algorithm as an evolutionary algorithm consumes more time than the other local-search-based algorithms in most instances. The weakness of the local search capability of the memetic algorithm is more evident in large-scale instances because the best objectives it found are not as good as those by RVND-SA. Therefore, it can be concluded that the proposed RVND-SA achieves the best balance between the solution quality and time efficiency, which could generate high-quality results within an acceptable computation time.

To further compare the performance of the proposed RVND-SA and each of its competitors, the Mann-Whitney U test has been applied, which is a non-parametric statistic hypothesis test used to compare two independent samples of ordinal data. We made a hypothesis for RVND-SA and each competitor on every instance as follows, where X represents one of the competitors. $Mean_X$ and $Mean_{RVND-SA}$ represent the average objective obtained X and RVND-SA.

$$H_0 : Mean_{RVND-SA} = Mean_X$$

$$H_a : Mean_{RVND-SA} < Mean_X$$

These hypotheses mean that RVND-SA outperforms another algorithm and is tested using one-sided tests. The null hypothesis can be rejected only if the p -Value is less than 0.05.

Furthermore, an indicator Gap is used, which can be calculated by $Gap_X = (Best_X - Best_{RVND-SA}) / Best_{RVND-SA}$, where $Best_X$ and $Best_{RVND-SA}$ represent the best objective obtained by X and RVND-SA.

Table 8 shows the p -value and Gap of all the solutions in 30 independent runs for each instance, where p -Value less than 0.05 and negative Gap are expressed in **bold**. The smaller the p -Value, the greater the confidence that the null hypothesis is rejected, i.e., the greater the confidence that RVND-SA outperforms competitor X . According to the results in Table 8, we can draw the following conclusions. For each

Table 9

The comparison results under different speeds of drones.

Instances	Algorithms	Speed of drones				
		25	30	35	40	45
SD151–12	MA	2.53	2.23	1.71	1.66	1.59
	VND	2.48	2.29	1.68	1.56	1.39
	VND-SA	2.52	2.08	1.71	1.59	1.41
	RVND	2.36	1.89	1.50	1.53	1.33
	RVND-SA	2.33	1.99	1.35	1.39	1.18
SD254–20	MA	4.67	4.56	4.62	3.82	3.35
	VND	4.44	4.23	3.45	3.62	3.14
	VND-SA	4.04	4.14	3.12	3.21	3.08
	RVND	4.18	3.99	3.41	3.41	3.19
	RVND-SA	3.92	3.59	2.84	2.81	2.98
SD295–30	MA	7.45	7.34	7.33	6.83	6.74
	VND	7.43	7.11	6.78	6.55	6.64
	VND-SA	7.39	7.28	7.07	6.42	6.25
	RVND	7.04	6.83	6.39	6.28	6.36
	RVND-SA	6.99	6.53	6.10	6.10	6.02

competitor, there are no less than 30 instances that support RVND-SA outperforming it. According to the p -values, MA performs better than RVND-SA on a few instances but is considered worse on all with more than seven target arcs, which can be explained by the fact that MA's global search has certain advantages only in relatively small-scale instances. As for VND, VND-SA, and RVND, only at most three instances each have p -Values greater than 0.05. In terms of the quality of the solution found, according to *Gaps*, MA, VND, and VND-SA outperform RVND-SA on at most two instances, while RVND, as its prototype algorithm, outperforms it on four.

5.4. Comparison of the algorithms under different speeds of drones

To analyze the effects of the speed of drones, the following experiments are conducted. All experiments are based on three instances with different scales in the transportation network Sydney, which are SD151–12, SD254–20, and SD295–30, respectively. Similarly, each algorithm is run 30 times on each instance. The comparison experiments are carried out by changing the speed of a drone gradually from 25 km/h to 45 km/h, with an interval of 5 km/h each step, while the speed of the GV is kept at 30 km/h. The results of all algorithms are reported in Table 9 and plotted Fig. 13, respectively.

Fig. 13(a), (b), and (c) show the results of the three instances with different scales varying from 12 to 30, respectively. It can be observed that with the increase of drone speed, the required total time for finishing all patrol tasks exhibits a downward trend. When the speed of the drone increases from 25 km/h to 45 km/h, the total patrol time obtained by RVND-SA on each instance is decreased by 1.14, 0.94, and 0.97 h, respectively. Especially when the drone speed increases from 30 km/h to 35 km/h, the required total patrol time drops the most. However, the

decrease rate of the total patrol time is getting slower as the speed of drones exceeds 35 km/h.

5.5. Comparison of the algorithms under different numbers of drones

The three instances, SD151–12, SD254–20, and SD295–30 are also used to test the impact of the number of drones carried by a ground vehicle. Each algorithm is run 30 times on each instance repeatedly. Comparison experiments are carried out by increasing the number of drones from 2 to 6. The results of all algorithms are summarized in Table 10 and depicted in Fig. 14, respectively.

The results of the three instances with different scales are illustrated in Fig. 14(a), (b), and (c), respectively. The impact of the number of drones on the final solution is obvious. As Fig. 14 displays, as the number of drones increases, the total patrol time decreases first and then remains relatively flat. It shows that the coordination of the GV with more drones will require more waiting time. It can be explained that more drones can bring more flights, which could reduce the overall flight time to offset part of the waiting time, as well as an increase in the scale of search space and a decrease in the efficiency of algorithms. The choice of the appropriate number of drones is therefore very important to reduce total patrol time and operation cost regarding to the drone platforms.

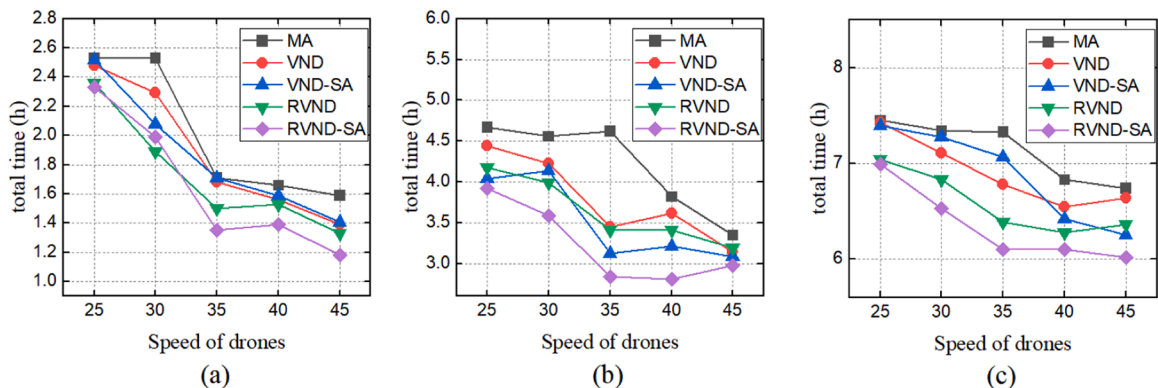
5.6. Case study

In order to further test the applicability of the proposed model and solution method, a practical application scenario is built based on the part of the practical road network in Changsha city, China. The urban road traffic area that we are interested in is shown in Fig. 15. The roads are represented by line segments. Particularly, 56 road intersections are

Table 10

The comparison results under different numbers of drones.

Instances	Algorithms	Speed of drones				
		2	3	4	5	6
SD151–12	MA	2.04	1.71	1.66	1.70	1.70
	VND	2.04	1.68	1.53	1.51	1.58
	VND-SA	1.85	1.71	1.50	1.52	1.49
	RVND	1.85	1.50	1.44	1.46	1.47
	RVND-SA	1.81	1.35	1.36	1.38	1.41
SD254–20	MA	4.60	4.62	3.91	3.98	3.85
	VND	3.52	3.45	3.52	3.40	3.57
	VND-SA	3.37	3.12	3.36	3.32	3.48
	RVND	3.57	3.41	3.21	3.21	3.20
	RVND-SA	3.38	2.84	3.09	3.20	3.05
SD295–30	MA	7.65	7.33	6.96	6.65	6.63
	VND	7.26	6.78	6.72	6.70	6.98
	VND-SA	7.70	7.07	6.98	6.93	7.09
	RVND	7.25	6.39	6.24	6.34	6.00
	RVND-SA	6.39	6.10	6.15	6.05	6.24

**Fig. 13.** The comparison results under different speeds of drones on SD151–12, SD254–20, and SD295–30.

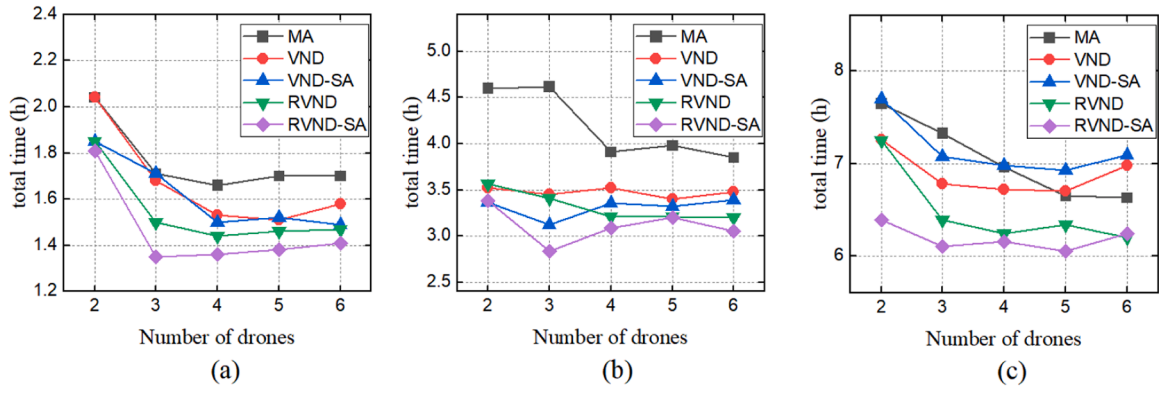


Fig. 14. The comparison results under different numbers of drones on SD151-12, SD254-20, and SD295-30.



Fig. 15. Patrol targets setting.

labeled in black, 94 road sections are represented by gray line segments, and 10 target arcs are marked by thick red lines. The blue dot represents the patrol center. Further, to ensure a smooth patrol, the flight endurance is set to 0.5 h.

The final solution in the map obtained by RVND-SA is shown in Fig. 16. The solid line represents the vehicle route while the dashed lines with different colors represent the different routes of the drones. As shown in Table 11, the total time of traffic patrol in the practical road network is 4956 s, and the CPU runtime of the algorithm is 37.29 s. Taking the patrol time obtained by RVND-SA as benchmark 1, the patrol times of MA, VND, VND-SA and RVND can be expressed as 1.05, 1.09, 1.05 and 1.03, which are worse than that of RVND-SA. According to the results of the real-world case, it can be found that the patrol route scheme can be efficiently obtained by RVND-SA. The results for the other algorithms are shown in Fig.A.1-4 (in the appendix).

6. Conclusion

In this paper, for the first time, a ground vehicle and multiple drones is formed an air-ground inspection system to cooperatively realize the agile urban traffic patrol. The routing problem of this inspection system,

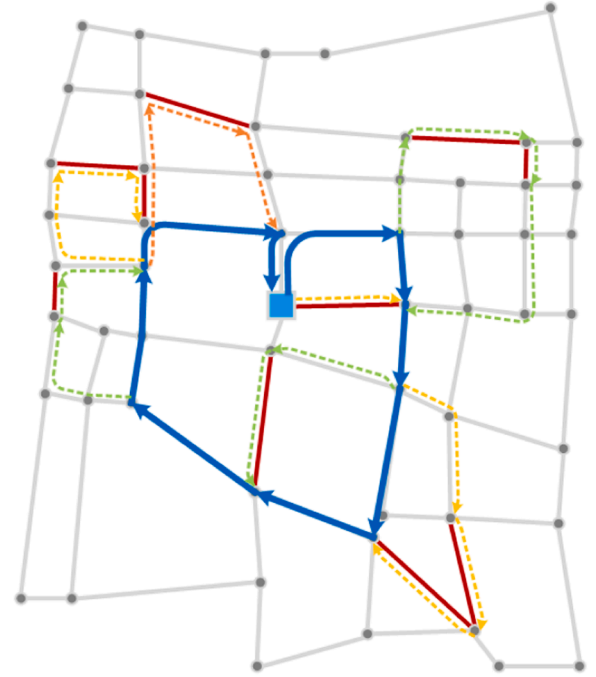


Fig. 16. The result of the practical application scenario in Changsha for RVND-SA.

Table 11
Results of case study on different algorithms.

	Results	MA	VND	VND-SA	RVND	RVND-SA
Total time (s)		5215	5391	5227	5120	4956
Relative value		1.05	1.09	1.05	1.03	1.00
CPU time (s)		41.89	28.11	27.47	40.93	37.29

which is described as a GV-drone arc routing problem (GVD-ARP), is particularly investigated. A mathematical model is established to describe GVD-ARP and an effective metaheuristic algorithm combining SA with RVND (RVND-SA) is proposed to solve the problem. In RVND-SA, the Metropolis criterion for solution selection and four problem-specific neighborhood search strategies are adopted to find a high-quality solution. The proposed model and algorithm are verified by a set of small-scale instances. Extensive numerical experiments are conducted to show the effectiveness of the algorithm. Meanwhile, sensitivity analyses on two key parameters of the drone are performed on instances with different scales. In addition, experiments on a real-world

case are carried out to test the applicability of the proposed model and method. The experimental results demonstrate that the proposed RVND-SA is efficient in solving the GV-drone arc routing problem. In the future, the routing problem about coordinating multiple GVs and multiple drones would be more complicated and deserves further investigation.

CRedit authorship contribution statement

Binjie Xu: Writing – original draft, Methodology, Software. **Kexin Zhao:** Visualization, Investigation, Writing – original draft, Data curation. **Qizhang Luo:** Resources, Validation. **Guohua Wu:** Conceptualization, Project administration, Writing – review & editing. **Witold Pedrycz:** Supervision, Writing – review & editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 62073341), and the Natural Science Fund for Distinguished Young Scholars of Hunan Province (Grant No. 2019JJ20026).

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.swevo.2023.101246.

References

- [1] Y. Lou, Y. Yin, S. Lawphongpanich, Freeway service patrol deployment planning for incident management and congestion mitigation, *Transp. Res. Part C Emerg. Technol.* 19 (2) (2011) 283–295, 2011/04/01/.
- [2] H. Zhang, G. Dai, The strategy of traffic congestion management based on case-based reasoning, *Int. J. Syst. Assurance Eng. Manage.* 10 (1) (2019) 142–147.
- [3] N.R. Sabar, A. Bhaskar, E. Chung, A. Turkey, A. Song, A self-adaptive evolutionary algorithm for dynamic vehicle routing problems with traffic congestion, *Swarm Evol. Comput.* 44 (2019) 1018–1027, 2019/02/01/.
- [4] R.M. Li, F.C. Pereira, M.E. Ben-Akiva, Competing risk mixture model and text analysis for sequential incident duration prediction, *Transp. Res. Part C Emerg. Technol.* 54 (May 2015) 74–85.
- [5] N.H. Motlagh, T. Taleb, O. Arouk, Low-altitude unmanned aerial vehicles-based internet of things services: comprehensive survey and future perspectives, *IEEE Internet Things J.* 3 (6) (2016) 899–922.
- [6] N.C. Coops, T.R.H. Goodbody, L. Cao, Four steps to extend drone use in research, *Nature* 572 (7770) (Aug. 2019) 433–435.
- [7] G. Macrina, L.D.P. Pugliese, F. Guerriero, G. Laporte, Drone-aided routing: a literature review, *Transp. Res. Part C Emerg. Technol.* 120 (2020), 102762.
- [8] M. Salama, S. Srinivas, Joint optimization of customer location clustering and drone-based routing for last-mile deliveries, *Transp. Res. Part C Emerg. Technol.* 114 (2020) 620–642.
- [9] H. Luo, P. Zhang, J. Wang, G. Wang, F. Meng, Traffic patrolling routing problem with drones in an urban road system, *Sensors* 19 (23) (2019).
- [10] Y. Liu, J. Shi, Z. Liu, J. Huang, T. Zhou, Two-layer routing for high-voltage powerline inspection by cooperated ground vehicle and drone, *Energies* 12 (7) (2019) 1385.
- [11] L. Jianyuan, L. Fang, W. Yue, and T. Yiping, "Optimizing traffic patrol based on generalized maximal covering model." pp. 59–63.
- [12] M. Saint-Guillain, C. Paquay, S. Limbourg, Time-dependent stochastic vehicle routing problem with random requests: application to online police patrol management in Brussels, *Eur. J. Oper. Res.* 292 (3) (2021) 869–885, 2021/08/01/.
- [13] X. Sun, J. Wang, Routing design and fleet allocation optimization of freeway service patrol: improved results using genetic algorithm, *Physica A* 501 (2018) 205–216, 2018/07/01/.
- [14] D.A. Steil, J.R. Pate, N.A. Kraft, R.K. Smith, B. Dixon, L. Ding, A. Parrish, Patrol routing expression, execution, evaluation, and engagement, *IEEE Trans. Intell. Transp. Syst.* 12 (1) (2010) 58–72.
- [15] S.S. Chawathe, "Organizing hot-spot police patrol routes." pp. 79–86.
- [16] D. Reis, A. Melo, A.L. Coelho, and V. Furtado, "Towards optimal police patrol routes with genetic algorithms." pp. 485–491.
- [17] P. Li, J.R. Walton, Evaluating freeway service patrols in low-traffic areas using discrete-event simulation, *J. Transp. Eng.* 139 (11) (Nov 1, 2013) 1095–1104.
- [18] R. Du, C. Chen, B. Yang, N. Lu, X. Guan, X. Shen, Effective urban traffic monitoring by vehicular sensor networks, *IEEE Trans. Veh. Technol.* 64 (1) (2014) 273–286.
- [19] Z. Wang, W. He, X. Zhang, Y. Wang, B. Wu, Y. Wang, Lane-based vehicular speed characteristics analysis for freeway work zones using aerial videos, *Can. J. Civ. Eng.* 48 (3) (2021) 274–283.
- [20] D. Wu, D.I. Arkhipov, M. Kim, C.L. Talcott, A.C. Regan, J.A. McCann, N. Venkatasubramanian, ADDSEN: adaptive data processing and dissemination for drone swarms in urban sensing, *IEEE Trans. Comput.* 66 (2) (2016) 183–198.
- [21] A.A. Munishkin, D. Milutinovi, D.W. Casbeer, Minimax time efficient inspection of ground vehicles by a UAV team, *Rob. Auton. Syst.* 125 (2020), 103370.
- [22] Á. Corberán, G. Laporte, Arc routing: problems, methods, and applications, *SIAM*, 2015.
- [23] M. Kwan, Programming method using odd or even pints, *Acta Mathematica Sinica* 10 (1960) 263–266.
- [24] V.H.R. Lima, E. de Oliveira Lima, H. Sherafat, Roteirization of vehicles in the delivery/collection problems-Application of a modified Ant Colony Algorithm, *Revista Brasileira de Computação Aplicada* 12 (1) (2020) 44–53.
- [25] C.H. Papadimitriou, On the complexity of edge traversing, *JACM* 23 (3) (1976) 544–554.
- [26] A. Corberán, R. Eglese, G. Hasle, I. Plana, J.M. Sanchis, Arc routing problems: a review of the past, present, and future, *Networks* 77 (1) (Jan, 2021) 88–115.
- [27] T. Calogiuri, G. Ghiani, E. Guerriero, R. Mansini, A branch-and-bound algorithm for the time-dependent rural postman problem, *Comput. Oper. Res.* 102 (2019) 150–157.
- [28] M. Monroy-Licht, C.A. Amaya, A. Langevin, Adaptive large neighborhood search algorithm for the rural postman problem with time windows, *Networks* 70 (1) (2017) 44–59, 08/.
- [29] A. Hertz, G. Laporte, M. Mittaz, Tabu search heuristic for the capacitated arc routing problem, *Oper. Res.* 48 (1) (2000) 129–135.
- [30] S.-H. Huang, P.-C. Lin, Multi-treatment capacitated arc routing of construction machinery in Taiwan's smooth road project, *Autom. Constr.* 21 (2012) 210–218.
- [31] L. Xing, P. Rohlfshagen, Y. Chen, X. Yao, An evolutionary approach to the multidrop capacitated arc routing problem, *IEEE Trans. Evol. Comput.* 14 (3) (2009) 356–374.
- [32] B. Cao, W. Zhang, X. Wang, J. Zhao, Y. Gu, Y. Zhang, A memetic algorithm based on two Arch2 for multi-depot heterogeneous-vehicle capacitated arc routing problem, *Swarm Evol. Comput.* 63 (2021), 100864, 2021/06/01/.
- [33] H. Oh, S. Kim, A. Tsourdos, B.A. White, Coordinated road-network search route planning by a team of UAVs, *Int. J. Syst. Sci.* 45 (5) (May 4, 2014) 825–840.
- [34] J.F. Campbell, Á. Corberán, I. Plana, J.M. Sanchis, Drone arc routing problems, *Networks* 72 (4) (2018) 543–559.
- [35] S. Poikonen, J.F. Campbell, Future directions in drone routing research, *Networks* 77 (1) (Jan, 2021) 116–126.
- [36] C.C. Murray, A.G. Chu, The flying sidekick traveling salesman problem: optimization of drone-assisted parcel delivery, *Transp. Res. Part C Emerg. Technol.* 54 (2015) 86–109.
- [37] N. Agatz, P. Bouman, M. Schmidt, Optimization approaches for the traveling salesman problem with drone, *Transport. Sci.* 52 (4) (2018) 965–981.
- [38] K. Dorling, J. Heinrichs, G.G. Messier, S. Magierowski, Vehicle routing problems for drone delivery, *IEEE Trans. Syst. Man, Cybern. Syst.* 47 (1) (2017) 70–85, 01/.
- [39] Q.M. Ha, Y. Deville, Q.D. Pham, M.H. Ha, On the min-cost traveling salesman problem with drone, *Transp. Res. Part C Emerg. Technol.* 86 (Jan, 2018) 597–621.
- [40] Y. Liu, Z. Liu, J. Shi, G. Wu, W. Pedrycz, Two-echelon routing problem for parcel delivery by cooperated truck and drone, *IEEE Trans. Syst. Man Cybern. Syst.* (2020).
- [41] A. Karak, K. Abdelghany, The hybrid vehicle-drone routing problem for pick-up and delivery services, *Transp. Res. Part C Emerg. Technol.* 102 (2019) 427–449, 2019/05/01/.
- [42] M. Hu, W. Liu, J. Lu, R. Fu, K. Peng, X. Ma, J. Liu, On the joint design of routing and scheduling for vehicle-assisted Multi-UAV inspection, *Future Generat. Comput. Syst.* 94 (2019) 214–223.
- [43] M.W. Ulmer, B.W. Thomas, Same-day delivery with heterogeneous fleets of drones and vehicles, *Networks* 72 (4) (2018) 475–505.
- [44] D. Schermer, M. Moeini, O. Wendt, A hybrid VNS/Tabu search algorithm for solving the vehicle routing problem with drones and en route operations, *Comput. Oper. Res.* 109 (2019) 134–158.
- [45] S.G. Manyam, D.W. Casbeer, and K. Sundar, "Path planning for cooperative routing of air-ground vehicles," *2016 American Control Conference (ACC)*. pp. 4630–4635.
- [46] Z. Luo, Z. Liu, J. Shi, A two-echelon cooperated routing problem for a ground vehicle and its carried unmanned aerial vehicle, *Sensors* 17 (5) (2017) 1144.
- [47] D. Martin, L. Gilbert, Improvements and extensions to the Miller-Tucker-Zemlin subtour elimination constraints, *Oper. Res. Lett.* 10 (1) (1991) 27–36.
- [48] P. Hansen, N. Mladenovic, J.A.M. Perez, Variable neighbourhood search: methods and applications, *Ann. Oper. Res.* 175 (1) (Mar, 2010) 367–407.
- [49] M.E.H. Sadati, B. Çatay, A hybrid variable neighborhood search approach for the multi-depot green vehicle routing problem, *Transp. Res. E Logist. Transp. Rev.* 149 (2021), 102293, 2021/05/01/.

- [50] E.W. Dijkstra, A note on two problems in connexion with graphs, *Numer. Math.* 1 (1) (1959) 269–271.
- [51] J.C. de Freitas, P.H.V. Penna, A randomized variable neighborhood descent heuristic to solve the flying sidekick traveling salesman problem, *Electronic Notes Discrete Math.* 66 (2018) 95–102.
- [52] J.C. de Freitas, P.H.V. Penna, A variable neighborhood search for flying sidekick traveling salesman problem, *Int. Trans. Oper. Res.* 27 (1) (2020) 267–290.
- [53] D. Priyom, B.S. Mahanand, Chapter 9 - Affordable energy-intensive routing using metaheuristics, *Cognitive Data Sci. Sustain. Comput.* (2022) 193–210.
- [54] L. Shengcai, T. Ke, Y. Xin, Memetic search for vehicle routing with simultaneous pickup-delivery and time windows, *Swarm Evol. Comput.* 66 (2021), 100927.
- [55] N. Mladenović, P. Hansen, Variable neighborhood search, *Comput. Oper. Res.* 24 (11) (1997) 1097–1100.
- [56] Transportation Networks for Research Core Team, *Transportation Networks for Research*. <https://github.com/bstabler/TransportationNetworks>, 2021 (accessed 5 July 2003).
- [57] R. Stefan, P. David, A unified heuristic for a large class of vehicle routing problems with Backhauls, *Eur. J. Oper. Res.* 171 (3) (2006) 750–775.